As soon as you start a Ruby program, you're sitting within an object named main that the Ruby interpreter created for you. This object is sometimes called the *top-level context*, because it's the object you're in when you're at the top level of the call stack: either you haven't called any method yet or all the methods that you called have returned. (Oh, and in case you're wondering, Ruby's main has nothing to do with the main() functions in C and Java.)

### Class Definitions and self

In a class or module definition (and outside of any method), the role of self is taken by the class or module itself.

```
class MyClass
  self        # => MyClass
end
```

This last detail is not going to be useful right now, but it will become a central concept later in this book. For now, we can set it aside and go back to the main topic.

Everything that you've learned so far about method execution can be summed up in a few short sentences. When you call a method, Ruby looks up the method by following the "one step to the right, then up" rule and then executes the method with the receiver as self. There are some special cases in this procedure (for example, when you include a module), but there are no exceptions…except for one.

## Refinements

Remember the first refactoring you coded today, in *Open Classes*, on page 11? You and Bill used an *Open Class (14)* to add a method to Strings:

object_model/alphanumeric.rb
```
class String
  def to_alphanumeric
    gsub(/[^\w\s]/, '')
  end
end
```

The problem with modifying classes this way is that the changes are global: from the moment the previous code is executed, every String in the system gets the changes. If the change is an incompatible *Monkeypatch (16)*, you might break some unrelated code—as happened to you and Bill when you inadvertently redefined Array#replace.

*Spell: Refinement*

Starting with Ruby 2.0, you can deal with this problem using a *Refinement*. Begin by writing a module and calling refine inside the module definition: